

we would need to keep in mind that each virtual address will have a different number of sharable pages, and also account for the overhead of the KSM subsystem and data structures to track these shareable pages.

An overview of the design

In this article, we look at the salient design features of KSM. The reader is encouraged to read the paper titled *Increasing memory density by using KSM* [www.kernel.org/doc/ols/2009/ols2009-pages-19-28.pdf] and the kernel source code (*mm/ksm.c* under the kernel source directory). The salient features are:

1. **User hints:** The user provides hints on the address that can be considered as 'mergeable' using the *madvise(2)* system call. The KSM subsystem uses these hints to decide what virtual address space to scan for pages with duplicate content. Our code example will illustrate this API and show the correct usage of the call.
2. **Kernel thread:** KSM creates a kernel thread to help with merging pages with similar content in the background. This might change in the future, and KSM might create several threads depending on the topology of the system and the total available memory.
3. **Stability:** KSM maintains two data structures -- the stable and unstable tree. KSM works in multiple passes, checking to see if pages with similar content can be merged. At the end of each pass, the unstable tree is reinitialised. A pass is a full scan of areas registered via the *madvise(2)* API. Before moving on to the core algorithm, let us introduce a few more VM concepts.
 - **Anonymous page:** A page obtained via *mmap(2)* or *malloc(3)* with *MAP_ANONYMOUS*. These pages are not file backed and are saved on the swap device, when the system is under memory pressure.
 - **File backed page:** These are typically obtained via *mmap(2)* or read in and cached by the operating system when the *read(2)/write(2)* call is invoked. As the name suggests, these are pages that belong to a filesystem or a block device.
4. KSM is currently designed only to work with anonymous pages. This works very well for a KVM-based virtualisation environment, since all of the memory used by the virtual machine/OS is mapped into the qemu process. For each anonymous page scanned, the algorithm looks for a match in the stable tree. If a match is found, the page is merged with the page in the stable tree. We will look at what merging implies a little later in this article.
5. It is possible that no match is found, in which case we look at the checksum of the page (computed from the contents of the page) and see if it has changed since the last pass. If it has, then we consider the contents of this page as changing frequently and do not add it to the unstable tree. Otherwise, we add it to the unstable tree and look for pages with similar contents in the unstable tree. If a match is found, the pages are

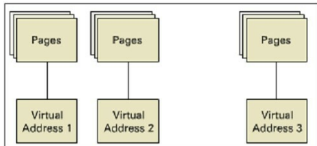


Figure 1: Several applications running in the hypervisor prior to KSM-enabled sharing

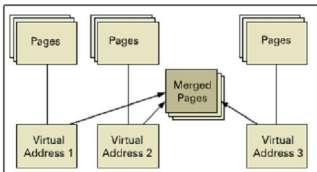


Figure 2: Several applications running in the hypervisor after KSM-enabled sharing

- merged and moved to the stable tree.
6. The tree data structure used is a red-black tree.
7. We referred to page merging earlier. The page-merging algorithm in KSM merges pages by sharing the page (making the page table entry the same); then it marks both entries as copy-on-write. The reason it does so is to make sure that if the contents change (one of the applications/virtual machines tries to change the contents by writing), the application that changes the page gets its own copy of the page.
8. **System parameters:** We use the following parameters to control KSM runtime behaviour.
 - *max_kernel_pages* – sets the maximum number of kernel pages that KSM can use
 - *pages_to_scan* – indicates how many pages the KSM thread will scan before going to sleep
 - *sleep_millsecs* – how long the KSM thread should sleep before starting a new scan/pass
 - *run* – Controls whether KSM should run. This value is an enumeration, so the user is referred to *Documentation/vm/ksm.txt* in the Linux source tree for further details.
9. KSM provides the following statistics:
 - *pages_shared* – the number of kernel pages KSM is using
 - *pages_sharing* – the number of pages being shared
 - *pages_unshared* – the number of pages that are not yet shared, but are being checked for duplicates
 - *pages_volatile* – the number of pages that are changing their contents too frequently
 - *full_scans* – how many times all merge-able areas (as hinted by *MADV_MERGEABLE*) have been scanned.